PRD: RSNA Knee MRI Abnormality Detection

Team: Olayiwola + teammate | **Version:** 1.0 | **Date:** 12 Oct 2026 | **Status:** Draft for team review

> **Scope update (12 Oct 2026): this is a 5-day hackathon build due Friday 16 Oct, not a Kaggle entry.** Section 17 is the plan that now applies. It supersedes the Kaggle deadlines, the leaderboard goals, the Kaggle submission requirements and the timelines in sections 12 and 16.4. Sections 1 to 11 stay as background and as the full-scope vision.

> **Verify before you build.** Items marked **[VERIFY]** come from secondary sources (news articles and community GitHub repos), not the Kaggle Overview/Rules/Evaluation tabs, which I could not read directly. Confirm each one against the Kaggle page before you commit to an architecture.

---

1. Overview

1.1 Problem

Knee MRI is the gold-standard exam for internal knee injuries, but findings such as ACL/MCL tears, meniscal damage and cartilage loss can be subtle, and radiologists do not always read them the same way. RSNA's 2026 challenge asks participants to build multimodal models that detect a defined set of clinically important abnormalities from knee MRI exams.

1.2 Confirmed competition facts

- Training data: **5,000+ knee MRI exams** from **16 sites** on five continents, each paired with its **original radiology report**, in **about 12 languages**.
- Evaluation data: a hidden set **annotated by expert radiologists**.
- It is RSNA's first challenge to combine images with report text.
- **$77,000** prize pool, with a separate **efficiency track**.
- Winners are announced at RSNA 2026.

1.3 Reported but unconfirmed **[VERIFY]**

- **12 independent binary findings** per exam (confirmed names so far: ACL, MCL, medial meniscus, lateral meniscus).
- Metric: **macro-averaged ROC-AUC**.
- **Code competition**: notebook runtime of up to about 9 hours, **internet disabled** at inference.
- Rule 4.b (data security) may restrict sending competition data to external APIs. Use open-weights models only.
- Labels for the training set may need to be **derived from the free-text reports** (weak labels), because the reports are not tabular.

1.3.1 Questions to settle on day 1

1. Do the training labels exist as a CSV, or must you extract them from reports?
2. Exactly which 12 findings, and are the outputs probabilities?
3. What are the efficiency-track rules (parameters, latency, memory)?
4. May competition data or sample images be shown in a public demo?

---

2. Product Vision and Goals

Vision: A trustworthy second-reader tool that takes a knee MRI exam (and optionally its report) and returns calibrated probabilities for each of the 12 abnormalities, with visual evidence a radiologist can check.

What this hackathon has to deliver (two layers):

Layer	Purpose	Priority
A. Competition model + submission notebook	Scores on the leaderboard. This is what the judges at Kaggle measure.	**P0**
B. Demo product (backend API + frontend viewer)	Shows the model as a usable clinical-style tool. This is what a hackathon audience sees.	**P1**

Be honest about the trade-off: the leaderboard only needs Layer A. Layer B is your presentation and differentiation layer. If time runs short, protect A, then ship a thin B.

2.1 Goals

- G1: Valid submission on the leaderboard that beats the public baselines (community baselines reported around 0.61 AUC **[VERIFY]**).
- G2: Calibrated, per-finding probabilities that hold up across sites.
- G3: A working demo: upload or select an exam, get 12 scores with slice-level evidence.
- G4: Reproducible, documented pipeline a teammate can run end to end.
- G5 (stretch): Efficiency-track submission (small, fast model).

2.2 Non-goals

- Clinical deployment or regulatory clearance.
- Diagnosing patients or replacing radiologists.
- Handling non-knee anatomy.
- Storing or redistributing competition data outside Kaggle.
- Training on external patient data.

---

3. Users and Personas

Persona	Need	How the product serves them
Radiologist (primary demo user)	Fast triage and a second opinion on subtle findings	Ranked findings, probability bars, key-slice viewer
Orthopedic or sports-medicine clinician	Quick sense of ligament and meniscus status	Summary card per structure
Hackathon judge	See technical depth plus real-world framing	Live demo, metrics page, architecture, limits stated openly
Teammate and future maintainer	Run, debug, extend	Clear repo layout, configs, docs

---

4. User Stories

1. As a radiologist, I upload (or pick) a knee MRI exam and see 12 abnormality probabilities within seconds.
2. As a radiologist, I scroll the MRI series (sagittal, coronal, axial) and see the slices that drove each prediction highlighted.
3. As a radiologist, I can set a threshold per finding and see which findings flip.
4. As a radiologist, I can paste the original report text and see whether the model and report agree or disagree.
5. As a judge, I open a metrics page showing per-finding AUC, per-site performance and calibration.
6. As a developer, I run one command to train, one to evaluate, one to produce a submission.

---

5. Scope

5.1 MVP (must ship)

- Data audit and label pipeline.
- Baseline image model (2.5D multi-plane) with 5-fold CV.
- Valid Kaggle submission notebook that runs offline within the time limit.
- Inference API with a single predict endpoint.
- Minimal frontend: exam picker, series viewer, 12 probability bars.

5.2 Should have

- Report-text branch (multilingual) fused with image features.
- Attention heatmaps or key-slice ranking.
- Metrics and calibration dashboard.
- Model ensemble.

5.3 Could have

- Efficiency-track variant (distilled or quantized).
- Report-to-image disagreement flagging.
- Exportable PDF summary.

5.4 Will not do

- User accounts, billing, EHR/PACS integration.
- Real PHI handling.

---

6. Functional Requirements

Priority key: **P0** must, **P1** should, **P2** could.

6.1 Data and Label Pipeline (backend/ML)

ID	Requirement	Pri
FR-D1	Ingest DICOM series per exam; read orientation, spacing, series descriptions, laterality	P0
FR-D2	Classify each series by plane (sagittal/coronal/axial) and sequence (PD, T2, T1, fat-sat) using metadata with image-based fallback	P0
FR-D3	Normalize intensities per series; resample or crop to a fixed grid; mirror right/left knees to a canonical orientation	P0
FR-D4	Obtain 12 binary labels per exam: use provided labels if present, otherwise derive from reports	P0
FR-D5	If deriving from reports: run an open-weights multilingual extractor (rules plus a local model) producing per-finding present/absent/uncertain	P1
FR-D6	Build cross-validation folds **grouped by site and patient/exam** to prevent leakage and measure cross-site generalization	P0
FR-D7	Cache preprocessed tensors to disk to keep training epochs fast	P1
FR-D8	Data audit report: class prevalence per finding, per site, per language, missing series, corrupt files	P0

6.2 Modeling

ID	Requirement	Pri
FR-M1	Baseline: 2.5D CNN (e.g. EfficientNet-class) over stacked adjacent slices, one encoder per plane, pooled features	P0
FR-M2	Multi-label head: 12 independent sigmoid outputs	P0
FR-M3	Loss handles class imbalance (weighted BCE or focal) and uncertain labels (masked or soft targets)	P0
FR-M4	Per-finding attention or MIL pooling over slices to identify key slices	P1
FR-M5	Report-text branch: multilingual encoder, fused late with image embedding. Must degrade gracefully when no report is supplied	P1
FR-M6	Post-hoc calibration per finding (temperature or isotonic) fit on out-of-fold predictions	P1
FR-M7	Ensemble across folds and seeds with test-time augmentation	P1
FR-M8	Efficiency variant: smaller backbone, lower resolution, optional distillation or quantization	P2
FR-M9	Experiment tracking: every run logs config, git hash, seed, fold metrics	P0

6.3 Evaluation

ID	Requirement	Pri
FR-E1	Compute macro ROC-AUC out-of-fold, plus per-finding AUC	P0
FR-E2	Report per-site and per-language AUC to expose distribution shift	P1
FR-E3	Calibration curves and Brier score per finding	P1

ID	Requirement	Pri
FR-E4	Error analysis notebook: top false positives and negatives with slices	P2

6.4 Submission

ID	Requirement	Pri
FR-S1	Single offline notebook: loads weights from a Kaggle Dataset, runs inference on the hidden test, writes the required submission file format **[VERIFY]**	P0
FR-S2	Runs inside the runtime limit with margin (target under 6 h if the limit is 9 h)	P0
FR-S3	No internet calls at inference	P0
FR-S4	Deterministic output given fixed seeds	P1
FR-S5	Dry-run mode on a small subset to validate the pipeline before a full run	P0

6.5 Backend API (demo layer)

ID	Requirement	Pri
FR-B1	`POST /v1/predict`: accepts an exam (zip of DICOM or a preloaded exam ID) and optional report text; returns 12 probabilities, calibrated scores, model version	P0
FR-B2	`GET /v1/exams` and `GET /v1/exams/{id}`: list demo exams and metadata	P0
FR-B3	`GET /v1/exams/{id}/series/{sid}/slices/{n}`: return a rendered slice image (PNG/WebP) with optional overlay	P0
FR-B4	`GET /v1/exams/{id}/explain?finding=ACL`: return key-slice indices and a heatmap	P1

ID	Requirement	Pri
FR-B5	`GET /v1/metrics`: serves precomputed evaluation results for the dashboard	P1
FR-B6	`GET /healthz` and `GET /readyz`	P0
FR-B7	Async job pattern for slow predictions: submit, poll status, fetch result	P1
FR-B8	Input validation: reject non-DICOM, oversized, or non-knee uploads with clear errors	P0
FR-B9	Every response carries model version and a not-for-clinical-use disclaimer field	P0

Suggested stack (a recommendation, your call): Python + PyTorch for training; FastAPI for the API (same language as the model, which avoids a serialization boundary); pydicom/SimpleITK for DICOM; Redis or an in-process queue for jobs; Docker for packaging. Your profile suggests you also work in Node/TypeScript; that is viable for a gateway, but keep model serving in Python.

6.6 Frontend

ID	Requirement	Pri
FR-F1	**Exam list page:** choose a demo exam or upload one	P0
FR-F2	**Viewer:** three-plane MRI viewer with slice scroll, window/level adjust, series switcher	P0
FR-F3	**Results panel:** 12 findings as ranked probability bars with color bands (low/medium/high) and numeric values	P0
FR-F4	Clicking a finding jumps the viewer to its key slices and shows the heatmap overlay	P1
FR-F5	Per-finding threshold slider with live positive/negative state	P1
FR-F6	Report panel: paste text, show agreement or disagreement with model output	P2

ID	Requirement	Pri
FR-F7	**Metrics page:** per-finding AUC, per-site AUC, calibration plots, model card	P1
FR-F8	Persistent banner: research prototype, not for diagnosis	P0
FR-F9	Loading, empty, and error states for every async action	P0
FR-F10	Keyboard navigation for the viewer (arrow keys for slices)	P2

6.7 Frontend Technology Stack

The frontend is a single-page React + TypeScript app built with Vite, with server-rendered slices in a canvas viewer for the MVP. It is chosen for speed of delivery for a two-person team in 10 days, not for novelty.

Concern	Choice	Why this one	Alternative (and why not)
Framework	**React 18+ with TypeScript (strict mode)**	Large ecosystem for medical viewers and charts; types catch API-contract mistakes early	Vue or Svelte: fine, but fewer imaging examples
Build tool	**Vite**	Near-instant dev server and HMR; zero config	Next.js: no SEO or SSR need here, adds complexity
Routing	**React Router**	Three pages only (exams, viewer, metrics)	TanStack Router: more power than needed
Server state	**TanStack Query**	Caching, retries, and polling for the async predict job (FR-B7)	Hand-written fetch and useEffect: bug-prone

Concern	Choice	Why this one	Alternative (and why not)
Client/UI state	**Zustand**	Tiny store for viewer state: slice index, window/level, selected finding, thresholds	Redux Toolkit: heavier than this app needs
Styling	**Tailwind CSS**	Fast iteration, dark theme by default (radiology convention)	CSS modules: slower to build with
Components	**shadcn/ui (Radix primitives)**	Accessible sliders, tabs, dialogs, tooltips out of the box (supports 10.6)	MUI: heavier, harder to give a clinical look
Image viewer (MVP)	**HTML canvas drawing WebP/PNG slices rendered by the API** (FR-B3)	No DICOM parsing in the browser, so no raw patient data reaches the client; simple and fast to build	Cornerstone3D: see stretch row below
Image viewer (stretch)	**Cornerstone3D** (`@cornerstonejs/core`, `tools`, `dicom-image-loader`)	Real MPR, window/level and scroll tools; the standard open-source choice	Skip unless the MVP viewer is done by day 6; it has a steep learning curve
Charts	**Recharts**	Probability bars, per-site AUC bars, calibration curves with little code	visx or Plotly: more control, more time
API client	**openapi-typescript generated from FastAPI's OpenAPI schema**, plus a thin fetch wrapper	One source of truth for request/response types; the frontend breaks at compile time when the API changes	Hand-typed interfaces: drift silently

Concern	Choice	Why this one	Alternative (and why not)
Upload and validation	**react-dropzone** + **zod**	Drag-and-drop zip upload; client-side size and type checks before the server's own checks (FR-B8)	Plain file input: poor UX
API mocking	**MSW (Mock Service Worker)**	Lets the frontend developer build against fake predictions before the model exists	Waiting for the real API: blocks the whole team
Testing	**Vitest + React Testing Library**, plus one **Playwright** smoke test	Fast unit tests; one end-to-end check that upload to results works	Full e2e suite: not worth it in 10 days
Quality	**ESLint + Prettier + `tsc --noEmit` in CI**, **pnpm**, Node 20 LTS	Consistent code between two people	None needed
Hosting	Static build on **Vercel, Netlify or Cloudflare Pages**, API on a separate container host	Cheap, quick deploys; needs CORS configured on the API	Serving the build from the API container with nginx: simpler if you have one host

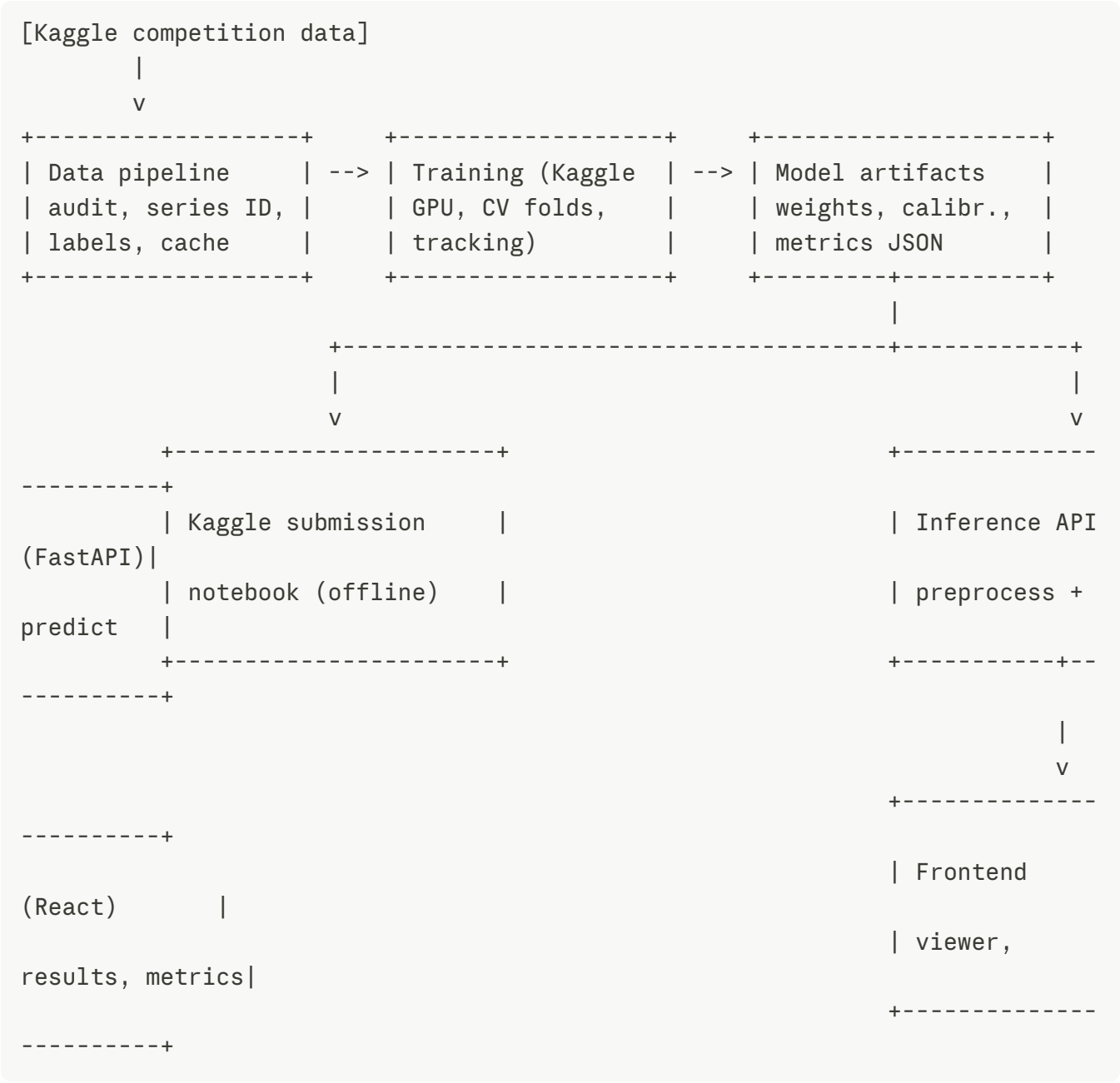
Viewer performance notes (NFR-P4, P5): prefetch the next few slices in the background, draw with `requestAnimationFrame`, send WebP rather than PNG, set long cache headers on slice URLs, and keep window/level adjustments client-side on the canvas so scrolling never waits on the server.

Decision to make together: the MVP canvas viewer is faster to ship but gives you no real DICOM tooling. Cornerstone3D looks more professional in a demo but can consume days. A reasonable rule: start with canvas, and move to Cornerstone3D only if the results panel and key-slice jumping are finished first.

Hosting caveat: do not deploy any competition images to a public site unless the rules allow it (see 10.4). Until confirmed, demo from a local machine or a recorded video.

---

7. System Architecture



Key design decisions

1. **One shared preprocessing module** used by training, the Kaggle notebook and the API. Divergence between these is the most common cause of a model that scores well in CV and badly in production.
2. **Training happens on Kaggle** (data stays there, GPU available). The demo API loads exported weights and can run on CPU for a handful of demo exams.
3. **Demo uses precomputed predictions as a fallback**, so a failed live inference never kills your presentation.

4. **Model outputs are versioned**; the API returns the model version with every prediction.

---

8. Suggested Repository Structure

```
rsna-knee/
  README.md                 # one-page quickstart
  docs/                     # PRD, data audit, decisions log, model card
  configs/                  # yaml per experiment
  src/
    data/                   # dicom loading, series classifier, labels, folds
    models/                 # encoders, heads, fusion
    training/               # loops, losses, calibration
    inference/              # shared preprocess + predict
    eval/                   # metrics, per-site analysis
  kaggle/                   # submission notebook + metadata
  api/                      # FastAPI app, schemas, routes
  web/                      # React app
  tests/                    # unit + smoke tests
  scripts/                  # train.sh, eval.sh, make_submission.sh
```

---

9. Data Model (demo layer)

- **Exam:** id, source site (if allowed), laterality, series[], optional report
- **Series:** id, plane, sequence type, slice count, spacing
- **Prediction:** exam id, model version, findings[12] {name, probability, calibrated, key_slices[]}
- **Metrics snapshot:** model version, per-finding AUC, per-site AUC, calibration bins

---

10. Non-Functional Requirements

10.1 Performance

ID	Requirement	Target
NFR-P1	Offline submission end-to-end runtime	Well under the limit (aim for 6 h or less on a 9 h limit) **[VERIFY]**

ID	Requirement	Target
NFR-P2	Demo prediction latency, preloaded exam	Under 5 s
NFR-P3	Demo prediction latency, fresh upload	Under 30 s, async with progress
NFR-P4	Slice render	Under 300 ms per slice
NFR-P5	Viewer frame feel when scrolling	No visible stutter on a normal laptop

These targets are my starting proposals; adjust once you measure real exam sizes.

10.2 Model quality

- Report out-of-fold macro AUC and per-finding AUC; do not tune on the public leaderboard alone.
- Track the gap between CV and leaderboard; a large gap signals site leakage or distribution shift.
- Calibration: Brier score reported per finding.

10.3 Reliability

- Training resumable from checkpoints (Kaggle sessions are time-limited and can die mid-run).
- API returns structured errors, never a stack trace.
- Graceful degradation: if the report branch or heatmap fails, still return image-only predictions.

10.4 Security and Privacy (important)

- Treat all data as sensitive. **Do not upload competition data to third-party APIs or LLM services** (Rule 4.b may prohibit this **[VERIFY]**).
- Do not redistribute competition data or host it publicly. Confirm whether sample images may appear in a public demo; if not, use a self-made or de-identified placeholder, or a video recording.
- Strip DICOM tags that could identify a patient before any display or logging.

- API: size limits, content-type checks, rate limiting, no persistent storage of uploads beyond a short TTL.
- Secrets in environment variables; never in the repo.

10.5 Compliance and Ethics

- Visible research-use-only disclaimer on every screen and API response.
- Model card documenting training data, known limits, per-site performance gaps and failure modes.
- Do not claim clinical validity.

10.6 Usability and Accessibility

- Usable on a standard laptop screen; dark theme by default (radiology convention).
- Colour is never the only signal (labels or icons beside colour bands).
- Keyboard-operable viewer controls (P2).

10.7 Maintainability and Reproducibility

- Fixed seeds, pinned dependencies, config-driven experiments.
- Single command each for train, evaluate, submit, serve.
- Smoke tests for preprocessing, shape contracts and API schema.
- Decision log in `docs/` explaining why, not only what.

10.8 Efficiency (if targeting the efficiency track) **[VERIFY rules]**

- Define and measure parameter count, inference time per exam and peak memory on the reference hardware.
- Maintain an accuracy-versus-cost table so trade-offs are visible.

10.9 Observability

- Structured logs with request id, model version, latency.
- Basic metrics: request count, error rate, p50/p95 latency.

---

11. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
Labels must be mined from multilingual reports and are noisy	High	High	Start with a rule-based extractor, spot-check manually, use soft or masked targets
Cross-site distribution shift hurts hidden-set score	High	High	Site-grouped CV, per-site metrics, intensity normalization, augmentation
Kaggle GPU session limits or crashes	Medium	High	Checkpoint and resume, cache preprocessing, keep a CPU-safe fallback
Notebook exceeds runtime limit	Medium	High	Dry-run early, budget inference time, fewer TTA passes
Rule violation (external API, data redistribution)	Medium	Severe	Read Rules tab on day 1; open-weights models only
Class imbalance and rare findings	High	Medium	Weighted loss, per-finding calibration, report per-finding AUC
Scope creep into a polished frontend	High	Medium	Freeze MVP UI by day 6; model quality has priority
Two-person team bottleneck	Medium	Medium	Clear ownership split (section 12)

---

12. Plan and Ownership (10 days to 22 Oct)

Suggested split: for a six-member team, use the breakdown in section 16; the dates below still apply.

Days	Milestone	Owner
12-13 Oct	Join competition, accept rules, read Rules/Evaluation tabs, resolve the four day-1 questions (1.3.1)	Both
13-14 Oct	Data audit, series classifier, label source decision	A
13-14 Oct	Repo scaffold, API skeleton, viewer shell with dummy data	B
15 Oct	**Entry deadline met; first valid baseline submission** (even if weak, proves the pipeline)	Both
16-18 Oct	Improve model: multi-plane, imbalance handling, CV, calibration	A
16-18 Oct	Real inference in API, results panel, key-slice viewer	B
19-20 Oct	Report branch or ensemble (pick one), efficiency variant if time	A
19-20 Oct	Metrics page, error states, disclaimer, demo fallback data	B
21 Oct	Full-length dry run of the submission notebook with margin; freeze	Both
22 Oct	Final submission, demo rehearsal, README and model card	Both

Rule of thumb: get an end-to-end weak baseline submitted first, then improve. A working pipeline at 0.6 beats a perfect model that never runs offline.

---

13. Success Metrics

Metric	Target
Valid leaderboard submission	Yes, before the entry deadline

Metric	Target
OOF macro AUC	Beat the community baseline; set a numeric goal after your first CV run
CV-to-leaderboard gap	Small and explainable
Demo works live	Preloaded exam returns results in under 5 s, with fallback
Reproducibility	Teammate reproduces results from README with no help

---

14. Open Questions

1. Exact label format and whether report-derived labels are needed.
2. Whether the report text is available at inference time on the hidden set (this decides whether the text branch can be used in the submission at all).
3. Efficiency-track metric and hardware reference.
4. Permitted use of sample data in a public demo.
5. Team compute budget outside Kaggle (for demo hosting).

---

15. Glossary

- **2.5D:** feeding a stack of neighbouring slices as channels to a 2D network.
- **MIL:** multiple-instance learning; the exam is a bag of slices with one label.
- **OOF:** out-of-fold predictions from cross-validation.
- **ROC-AUC:** area under the ROC curve; threshold-independent ranking quality.
- **Calibration:** whether a predicted 0.7 corresponds to about 70% observed frequency.

---

16. Six-Member Work Breakdown

Six people can work in parallel from day 1 if each owns one workstream and the handoffs between them are frozen as written contracts (16.3) by 13 Oct. Without those contracts, five people wait on one.

> **[VERIFY] Team size.** Kaggle competitions often cap team size (commonly 5). Check the Rules tab for this competition before 15 Oct. If six is not allowed, one member works as an unofficial contributor to the repo, and the Kaggle account that submits belongs to an official member.

16.1 Roles at a glance

#	Workstream	Owns	Main PRD sections	Hands off to
M1	Data engineering	DICOM ingest, series classifier, preprocessing, cache, folds, data audit	6.1	M2, M3, M4, M5
M2	Image modeling	2.5D baseline, training loop, loss, CV training, experiment tracking	6.2 (FR-M1 to M4, M9)	M4, M5
M3	Reports and NLP	Label extraction from multilingual reports, report-text branch, fusion	6.1 (FR-D4, D5), 6.2 (FR-M5)	M2, M4
M4	Evaluation and submission	OOF metrics, per-site analysis, calibration, ensemble, offline Kaggle notebook, runtime budget	6.2 (FR-M6, M7), 6.3, 6.4	M5, M6
M5	Backend and serving	FastAPI service, shared inference module, Docker, job queue, logging	6.5, 10.3, 10.9	M6
M6	Frontend	Viewer, results panel, metrics page, states, disclaimer, demo polish	6.6, 6.7	Everyone (demo)

M1 is the critical path for M2, M3 and M4, so M1 ships the simplest usable version first and improves it later.

16.2 Task breakdown per member

M1: Data engineering

1. Read DICOM headers for all exams; write the data audit (prevalence per site and language, missing series, corrupt files). *Output: audit report.*
2. Series classifier: plane and sequence from metadata, image fallback. *Output: `series_index` table.*
3. Preprocessing: intensity normalization, resample or crop to a fixed grid, mirror to canonical laterality. *Output: shared `preprocess()` function.*
4. Cache preprocessed tensors to disk. *Output: cached dataset on Kaggle.*
5. Site- and exam-grouped folds. *Output: `folds` file.*
6. Support: help M2 with data loader performance.

M2: Image modeling

1. Build a dummy-data training loop on day 1 so it is ready the moment M1's data lands. *Output: runnable `train` script.*
2. 2.5D multi-plane baseline with 12 sigmoid outputs. *Output: first checkpoint.*
3. Imbalance-aware and uncertain-label-aware loss (with M3's label format).
4. Attention or MIL pooling for key slices (feeds the viewer's heatmaps).
5. Resumable checkpoints; log every run with config, seed and fold metrics.
6. Backbone experiments in priority order, one change at a time.

M3: Reports and NLP

1. Inspect reports by language; write a rule-based finding extractor first. *Output: `labels` file with 1 / 0 / uncertain per finding.*
2. Hand-check a sample of 50 to 100 reports across languages to estimate extraction accuracy. *Output: error rate per language.*
3. Improve with an open-weights multilingual model (no external APIs; see 10.4).
4. Build the report-text branch and the late-fusion head. *Output: text embedding module with a clean off switch.*
5. Confirm with M4 whether report text is available at inference on the hidden set; if it is not, drop the text branch from the submission and keep it for labels only.

M4: Evaluation and submission

1. Metrics library: macro AUC, per-finding AUC, per-site AUC, Brier score. Write it against dummy predictions on day 1. *Output: `eval` module.*
2. Read the Evaluation tab; pin down the exact submission format. *Output: format spec the team agrees on.*

3. Build the offline submission notebook that loads weights from a Kaggle Dataset and runs inference with internet off. *Output: dry-run notebook on 15 Oct.*
4. Calibration per finding, then ensemble and test-time augmentation.
5. Measure runtime per exam and set the ensemble size from the runtime budget.
6. Export a metrics JSON snapshot for the dashboard (16.3, contract 6).

M5: Backend and serving

1. Day 1: publish the OpenAPI schema and a stub API returning fake predictions so M6 is never blocked. *Output: running stub.*
2. Wrap the shared `preprocess()` and model loader behind `predict()`.
3. Slice-rendering endpoint (WebP) and exam listing.
4. Async job flow, upload validation, structured errors, health checks.
5. Heatmap and key-slice endpoint using M2's attention output.
6. Dockerfile, logging with request id, and a one-command local run.

M6: Frontend

1. Scaffold with Vite, TypeScript and Tailwind; configure MSW from M5's schema. *Output: app shell with fake data on day 1.*
2. Exam list and upload page.
3. Canvas slice viewer with scroll and window/level.
4. Results panel: 12 probability bars, ranked, with disclaimer banner.
5. Finding to key-slice jump, threshold sliders, heatmap overlay.
6. Metrics page from M4's JSON; loading, empty and error states throughout.

16.3 Interface contracts (freeze by 13 Oct)

Each contract is one short file in `docs/contracts/`. Changing one needs a message to every consumer.

#	Contract	Producer	Consumers	Shape
1	**Finding list**	M3	All	Ordered list of the 12 finding names, fixed once **[VERIFY]** against the Kaggle page
2	**Labels file**	M3	M2, M4	One row per exam id; 12 columns with 1, 0 or missing for uncertain

#	Contract	Producer	Consumers	Shape
3	**Folds file**	M1	M2, M4	Exam id, fold number, site
4	**Preprocessed exam**	M1	M2, M4, M5	Per-plane tensor with fixed grid, dtype and orientation; same function in training, notebook and API
5	**Model I/O**	M2	M4, M5	Input: preprocessed exam (+ optional report text). Output: 12 probabilities plus optional per-slice attention
6	**Metrics JSON**	M4	M5, M6	Per-finding AUC, per-site AUC, calibration bins, model version
7	**OpenAPI schema**	M5	M6	Routes in 6.5; types generated for the frontend

16.4 Timeline and gates

Gate	Date	Pass criteria	Who must be ready
G0 Kickoff	13 Oct	Everyone joined and rules accepted; contracts 1 to 7 drafted; repo scaffold pushed; day-1 questions in 1.3.1 answered	All
G1 First submission	15 Oct	Baseline model (even weak) runs through the offline notebook and gives a valid score	M1, M2, M4
G2 Integration alpha	18 Oct	Real model behind the API; UI shows real probabilities for a demo exam	M2, M5, M6
G3 Feature freeze	20 Oct	Only bug fixes after this; efficiency variant decided in or out	All
G4 Full dry run	21 Oct	Submission notebook runs end to end with runtime margin; demo rehearsed once	M4, M5, M6
G5 Final submission	22 Oct	Submission made; README, model card and demo recording done	All

16.5 Rules of working together

- **Daily 15-minute stand-up** at a fixed time: what shipped, what is blocked, what you need from whom.
- **Branches and PRs:** one branch per task, a pull request reviewed by a member from a different workstream; no direct pushes to `main`.
- **Stub first:** every producer publishes a fake version of its output on day 1 so consumers can build in parallel.
- **GPU is the shared bottleneck.** Kaggle limits GPU hours per account per week (check the current figure). Agree who trains what, and log every run in one shared experiment sheet so nobody repeats work.
- **One shared preprocessing function** (contract 4). Nobody reimplements it.
- **Cross-help rule:** a member who is blocked for more than two hours says so at once, and the nearest workstream helps: M1 with M2, M3 with M4, M5 with M6.
- **Decision log:** write down why you chose something in `docs/decisions.md`; two lines each.
- **Demo owner:** M6 owns the live demo, M5 is on call; always keep precomputed results as a fallback.

16.6 Where this plan is most likely to break

Risk	Why	Early warning	Response
M1 is late	Three workstreams wait on it	No cached data by end of 14 Oct	M2 trains on a small subset; M3 and M4 work on stubs
Labels are noisy across languages	M3's extractor is weak in some languages	Per-language error rate above what you can accept after the hand-check	Use masked loss for uncertain labels; drop the text branch from the submission
Contract drift	Six people change formats unnoticed	Integration fails at G2	Contracts live in one folder; changes announced at stand-up
GPU quota runs out	Many experiments, shared quota	Idle at mid-week with unused ideas	Prioritize experiments by expected gain; one person runs training queue

Risk	Why	Early warning	Response
Demo breaks on the day	Live inference fails	Latency above target at G4	Show precomputed results; keep the recorded demo

16.7 Questions for the team

1. Who in the team has the most Kaggle GPU quota left this week?
2. Who is strongest in NLP, and who in frontend? Assign M3 and M6 to them first, then fill the rest.
3. Is the hackathon judged on the leaderboard, the demo, or both? This decides how much time M5 and M6 get versus M2 and M4.

---

17. Five-Day Hackathon Plan (Mon 12 to Fri 16 Oct)

We build a working knee-MRI triage demo in five days: pick an exam, see per-finding probabilities, jump to the slices behind each one. The Kaggle competition is the inspiration, not the target, so nothing here is scored on its leaderboard. Success is a demo that runs end to end, honest held-out numbers, and a team that can explain every choice.

17.1 What changes from sections 1 to 16

Item	Before	Now
Deadline	22 Oct, Kaggle	**Fri 16 Oct**, hackathon (exact hour to confirm)
Success measure	Leaderboard macro AUC	Working demo, plus held-out AUC reported honestly
Kaggle offline notebook, runtime limit, submission format (6.4, NFR-P1, M4 tasks 2, 3, 5)	P0	**Dropped**
Findings predicted	12	**4: ACL, MCL, medial meniscus, lateral meniscus** (extend only if time is left)

Item	Before	Now
Report-text branch and fusion (FR-M5)	P1	**Dropped.** Reports are used only to derive labels, and only if the dataset needs it
Ensemble, test-time augmentation, efficiency track (FR-M7, FR-M8)	P1 / P2	**Dropped**
Calibration (FR-M6)	Per-finding, isotonic or temperature	Temperature scaling only, if time
Async jobs (FR-B7), Cornerstone3D, Playwright test	P1 / stretch	**Dropped**
Arbitrary upload (FR-F1, FR-B8)	P0	Bundled demo exams first; upload only if time
Timelines in sections 12 and 16.4	10 days	Replaced by 17.5

17.2 Decide today: which data can we use?

The whole plan depends on this, so settle it by Monday evening.

> **Data-use warning.** Kaggle competition data is normally licensed for use in that competition. Using it in a separate hackathon may breach the competition's rules. Read the Rules section on data access and use today. RSNA's announcement listed informatics@rsna.org for questions; asking costs one email.

Option	What it is	Catch
A. Competition data	5,000+ exams, 16 sites, paired reports	Use may be restricted to the competition; very large to download; labels may need extracting from multilingual reports **[VERIFY]**
B. A public knee MRI research set (for example MRNet from Stanford)	Smaller set with abnormal, ACL tear and meniscal tear labels **[VERIFY]**	Needs registration and a signed agreement, which may take hours or days; likely no MCL label

Option	What it is	Catch
C. Data the hackathon organizers provide	Whatever they allow	Ask the organizers today
D. No trainable data in time	Build the full app on one or two sample exams with **clearly labelled placeholder predictions**	Not a real model; never present placeholder numbers as results

Start with whichever option you can access fastest, and keep D as the safety net so the app is never blocked by data.

17.3 Scope for Friday

Must ship (P0)

- Data pipeline for a subset (target 500 to 1,000 exams if using A or B).
- One 2.5D baseline model, trained once properly, predicting the 4 findings.
- Held-out evaluation: AUC per finding, with the numbers shown as they are.
- FastAPI service: exam list, slice images, `predict` (synchronous).
- Frontend: exam list, canvas slice viewer, results panel with probability bars, disclaimer banner.
- 3 to 5 bundled demo exams with precomputed predictions as a fallback.
- README and a one-page model card.

Should ship (P1)

- Click a finding to jump to its key slices with a heatmap.
- Metrics page.
- Threshold slider per finding.

Will not do

- Everything marked Dropped in 17.1, plus accounts, hosting beyond a simple deploy, and any clinical claim.

Expect a modest model. A community baseline for the full competition was reported near 0.61 AUC **[VERIFY]**, and this task is hard. A weak but honest number with a polished, well-explained pipeline is a good hackathon result; an inflated number is not.

17.4 Roles (six members; merge if fewer)

The six workstreams from section 16 stay, adjusted for the smaller scope. If you are fewer than six, merge M1 with M3, M2 with M4, and M5 with M6.

#	Workstream	Friday deliverable
M1	Data engineering	Cached subset, `preprocess()`, folds or train/val/test split, 3 to 5 demo exams
M2	Image modeling	Frozen checkpoint and its training config
M3	Labels and explainability	Labels file (extracted from reports only if option A), key-slice and heatmap output
M4	Evaluation and demo data	Held-out metrics JSON, model card, precomputed demo predictions, README
M5	Backend	FastAPI service running locally and in Docker, with the stub replaced by the real model
M6	Frontend	Viewer, results panel, metrics page, polished demo

The seven interface contracts in 16.3 still apply, with one change: the contract for the finding list is now the 4 findings. Freeze them tonight.

17.5 Day-by-day plan

Member	Mon 12	Tue 13	Wed 14	Thu 15	Fri 16
M1	Get data access, inspect, choose the subset, start download	Series classifier, `preprocess()`, cache the subset	Split files, loader speed, help M2	Fix data bugs, bundle demo exams	Spare capacity, QA
M2	Training loop running on dummy data	Baseline training on the first cached batch	Main training run (2 to 4 hours), save checkpoint	One improvement round, freeze best checkpoint by 3 pm	No new training
M3	Check label source; write the labels file	Finish labels; start attention or Grad-CAM code	Key slices and heatmaps from the first checkpoint	Heatmaps for all demo exams	QA, help demo

Member	Mon 12	Tue 13	Wed 14	Thu 15	Fri 16
M4	Metrics module on dummy predictions; write the contracts	Evaluate the baseline	Held-out metrics JSON; temperature scaling if time	Model card; precompute demo predictions	README, demo script, rehearsal
M5	OpenAPI schema and stub API	Exam list and slice-rendering endpoints	Real `predict()` with the first checkpoint	Error handling, Docker, load check	On call for the demo
M6	Scaffold, MSW from the API schema	Exam list and canvas viewer	Results panel on the real API	Finding to slice jump, metrics page, all states	Polish and own the demo

Gates

Gate	When	Pass criteria
G0 Foundations	Mon night	Data option chosen; contracts frozen; stub API running; UI scaffold pushed
G1 Fake end to end	Tue night	Cached data exists; UI shows stub predictions from the stub API
G2 First real model	Wed night	A real checkpoint answers through the API and appears in the UI
G3 Freeze	Thu 3 pm	Model and features frozen; only bug fixes after this; first full demo rehearsal Thursday evening
G4 Submit	Fri morning	Final rehearsal done, README and model card finished, submitted before the deadline

With five days, Thursday night is the real deadline. Friday is a buffer, not working time.

17.6 Demo script (3 minutes)

1. Pick a bundled exam and open the viewer; scroll through the three planes.
2. Show the 4 probability bars and the disclaimer banner.
3. Click ACL: the viewer jumps to its key slices with a heatmap.
4. Open the metrics page and state the held-out AUC per finding honestly, including the weak ones.

5. Name the limits: small training subset, research prototype, not for diagnosis.

Keep a recorded run of this as a fallback in case the live demo fails.

17.7 Risks specific to five days

Risk	Early warning	Response
Data access takes too long	No downloadable data by Mon night	Switch to option B or C; build on option D samples meanwhile
Training too slow or model near chance	Baseline AUC close to 0.5 after the first full run	Lower resolution, fewer slices, simpler backbone; report honestly
GPU hours run out	Quota warnings by Wed	One person owns the training queue; no parallel duplicate runs
Integration breaks late	G2 missed	Contracts frozen Mon; stub-first rule from 16.5
Scope creep	Anyone adds a feature after G3	Freeze rule; the Dropped list in 17.1 stays dropped
Live demo fails	Latency or crash during rehearsal	Precomputed predictions and a recorded demo

17.8 Questions to answer today

1. What exactly must be submitted on Friday (repo, live link, video, slides) and by what hour?
2. How is it judged: model quality, product and demo, or the idea?
3. Which data option from 17.2 can we actually access tonight?
4. How many people are on the team? If it is fewer than six, who takes which merged role?